

SOCRA

Data Architecture, Learning Intelligence & Model Improvement Pipelines

A technical design paper for the university research pilot and future AI-native LMS

Core thesis: Socra should treat student data as an auditable stream of educational evidence. Raw events, inferred learning states, faculty aggregates, research datasets, and model-training data must be separate products of the pipeline with explicit transformations and permissions between them.

Research snapshot: October 6, 2026

Scope: CSE 115 / CSE 116 pilot, expandable to a future multi-course platform

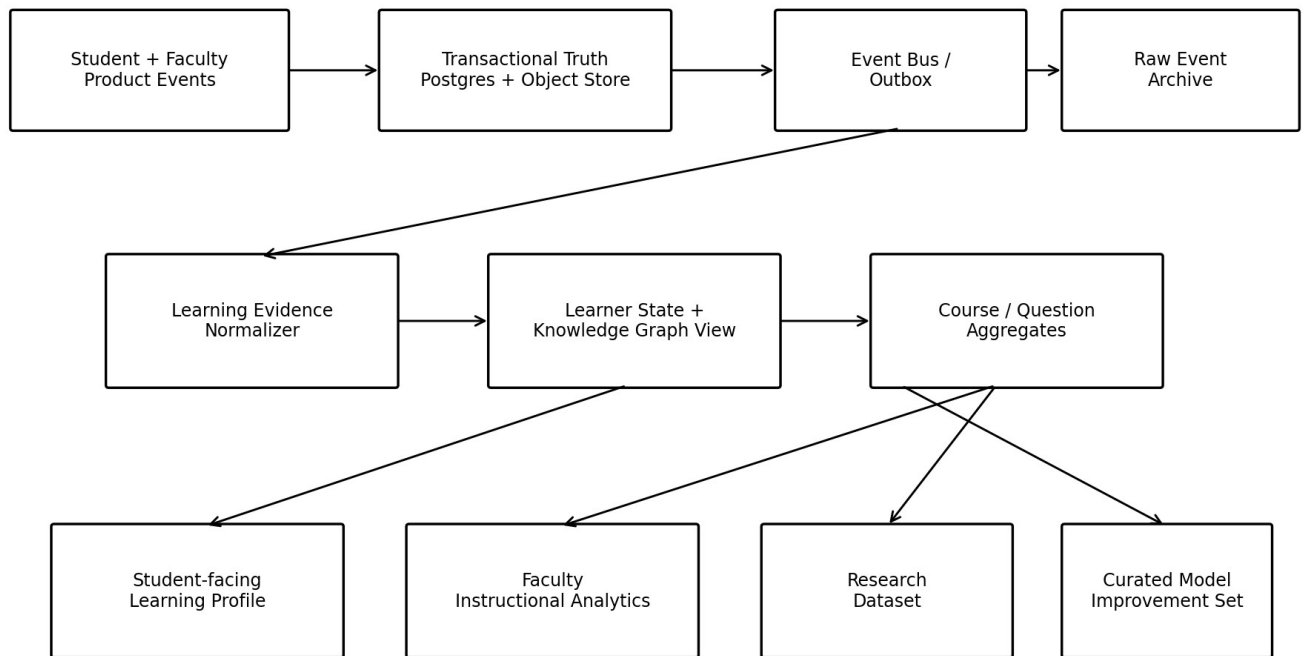
1. Executive Summary

Data is one of Socra's central product assets, but only if the architecture preserves meaning, provenance, privacy, and reproducibility. The system will observe more than final grades: assignment attempts, code runs, errors, Socratic interactions, intervention depth, practice outcomes, misconceptions, question difficulty, timing, course resources, submission outcomes, and faculty actions. The temptation is to pour all of this into a single analytics database or a single opaque student embedding. That would make the system difficult to audit and scientifically weak.

The recommended architecture separates data into four planes:

- Transactional truth: authoritative records such as users, courses, assignment versions, submissions, grades, code runs, and AI sessions.
- Learning evidence: normalized, versioned observations that say what happened and which concept it may provide evidence about, without claiming permanent ability.
- Analytics and research: privacy-controlled aggregates, pseudonymous datasets, metric tables, and reproducible feature snapshots derived from the evidence layer.
- Model improvement: a deliberately curated dataset created only after purpose, consent/protocol, de-identification, quality, and train/evaluation leakage gates.

Socra data architecture: separate truth, evidence, analytics, and training planes



Do not let model-generated labels become source-of-truth records. A misconception detector produces an observation with confidence and model version. It does not rewrite history or permanently label the student.

2. Data Design Principles

Event first, state second: Store the evidence that caused a learner-state update. The current state is a derived view that can be recomputed when models or policies change.

Purpose limitation: Each field and dataset should have a declared use: product operation, student personalization, faculty instruction, research, security, or model improvement.

Provenance everywhere: Derived records carry source event IDs, assignment/question versions, prompt/model versions, feature code version, and timestamps.

Separate identity from research: Use internal user IDs operationally and pseudonymous participant IDs in research datasets whenever possible.

Aggregate before faculty display: Faculty dashboards should use class/question/topic aggregates first. Raw Socra transcripts are not ordinary analytics data.

No single opaque learner embedding as truth: Use interpretable per-topic features and evidence. Semantic vectors can supplement retrieval and similarity, but should not be the only student representation.

Training data is opt-in by pipeline, not by accident: Production logs never become fine-tuning examples merely because they exist.

Version the semantics: A metric definition, misconception taxonomy, learner-state estimator, or prompt can change. Store versions so historical results remain interpretable.

Minimize irreversible inference: Do not store sensitive or high-impact inferred traits unless the product/research purpose clearly requires them.

Research reproducibility beats cleverness: For the pilot, a simpler interpretable model with clean lineage is preferable to a sophisticated learner model that cannot be explained or recreated.

3. What Data Socra Produces

Domain	Examples	Authority / sensitivity	Primary consumers
Identity	name, university email, role, course membership	Authoritative; FERPA-linked	Auth, roster, admin
Assignment	prompt, topic tags, rubric, policy, versions	Authoritative course record	Student/faculty/AI context
Student work	draft, code snapshot, written response, submission	Authoritative educational record	Student, grading
Execution	run ID, language, compile/runtime status, public test results	Operational + learning evidence	Student, Socra, analytics
AI interaction	message, response, model, prompt version, tokens, intervention level	Sensitive conversational data	Student history, research if approved
Learning evidence	correctness, attempt, hint depth, misconception observation	Derived, auditable	Learner model, analytics
Learner state	topic proficiency state, confidence, trend, recency	Derived estimate	Student, practice engine
Faculty aggregate	topic difficulty, first-attempt correctness, recovery rate	Derived aggregate	Faculty
Research	pseudonymous events, outcomes, condition	Controlled derived dataset	Research team
Training/eval	curated ideal responses, preference labels, adversarial prompts	Separate governed dataset	Model improvement team

4. Event Collection Pipeline

Socra needs both transactional records and event telemetry. These are related but not interchangeable. A submission table answers "what is the current/final submitted artifact?" An event stream answers "what sequence of learning actions led to it?"

4.1 Recommended event envelope

```

event_id: UUID
event_name: socra.response_completed
event_time: UTC timestamp
schema_version: 1
actor_id: internal user UUID
research_participant_id: nullable pseudonymous ID
course_id / section_id
assignment_id / assignment_version
question_id / question_version
session_id
study_condition
app_version
properties: typed event-specific payload
source_trace_id
privacy_classification

```

4.2 Collection mechanics

- Write the authoritative transaction first or in the same database transaction as an outbox record.
- Use the transactional outbox pattern or equivalent to avoid the classic failure where a database write succeeds but the analytics event is lost.
- A background publisher reads the outbox and delivers events to the event pipeline. For a university pilot, this can be a Postgres outbox plus a worker; Kafka is not required simply because the architecture is event-driven.
- Consumers build learning evidence, aggregates, research tables, and cost/operational metrics asynchronously.
- Events should be idempotent. Downstream jobs should tolerate replay without double-counting.

4.3 Events worth collecting

Family	Representative events	Do not collect by default
Assignment	opened, question viewed, draft saved, submitted, closed	every keystroke
Coding	run requested/completed, compile error, runtime error, public tests	full terminal environment variables
Socra	session started, prompt sent, response completed, intervention level, retrieval used	hidden chain-of-thought or provider internals
Practice	item shown, answered, explanation requested, completed	mouse-movement surveillance
Learning	misconception observed, evidence recorded, topic state changed	unsupported personality/ability labels
Faculty	assignment published, analytics viewed, grade finalized	irrelevant browsing behavior
Research	condition assigned, outcome captured, export generated	direct identifiers in analysis table if not needed

5. The Learning Evidence Pipeline

The learning evidence layer is the semantic heart of Socra. It converts low-level events into claims of limited scope, such as "this response is evidence about recursion base cases" or "the student required intervention level 4 on a medium-difficulty item." It should not directly claim that a learner has or lacks mastery.

Field	Purpose
evidence_id	Immutable identifier
user_id	Operational student link
topic_id	Concept the evidence bears on
activity_id / question_id	Source context
evidence_type	correctness, hint, misconception, practice, transfer, etc.

value	Typed value, e.g. correct=true or hint_depth=4
weight	Policy/model-determined contribution
confidence	Confidence in inferred evidence
observed_at	When event happened
source_event_ids	Lineage
detector_version	Which classifier/rule produced it
assignment_version	Prevents semantic drift
study_condition	Research reproducibility

5.1 Examples

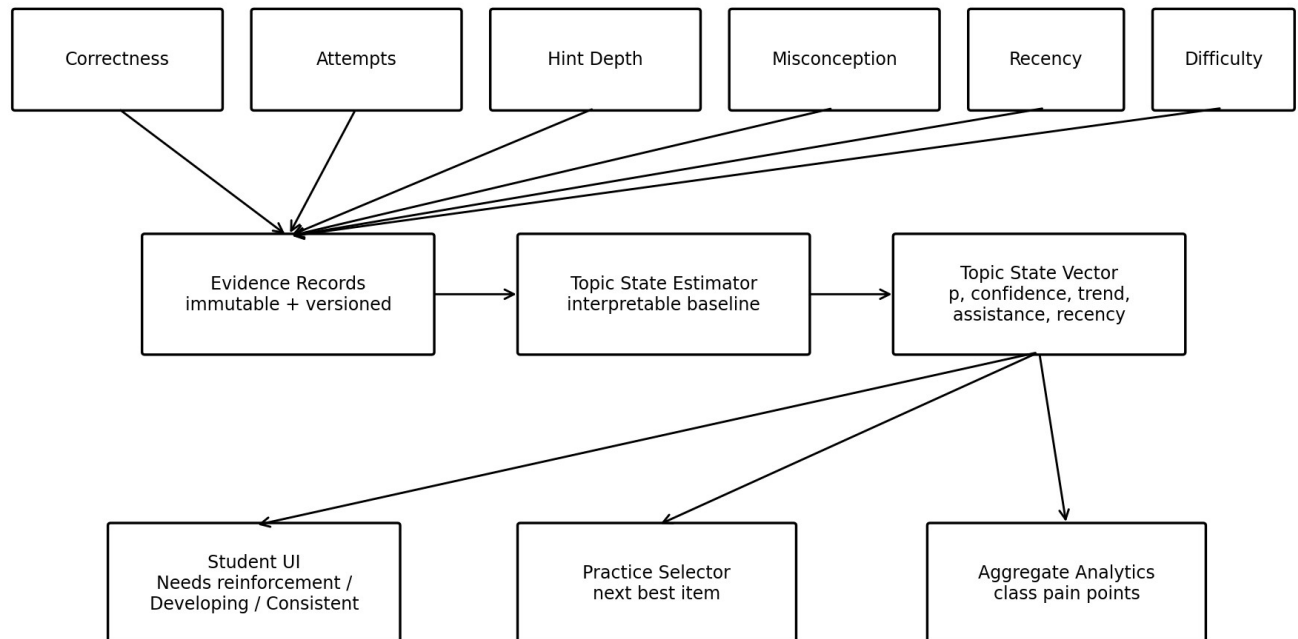
Raw event(s)	Normalized evidence
Question submitted correctly on first attempt	correctness=1; assistance_depth=0; context=protected
Student asks four Socratic questions before correcting code	assistance_depth=4; recovered_after_guidance=1
Model detects "all decreasing sequences reach zero" with 0.88 confidence	misconception_observation=decreasing_implies_zero; confidence=.88; detector=v3
Student answers a related practice problem correctly 10 days later	retention/transfer evidence with recency=10d

Keep raw evidence even after the student state changes. If Socra changes its learner-state algorithm next semester, the profile should be recomputable from historical evidence rather than locked to old scores.

6. Student Profile: Vector, Graph, and Knowledge Tracing

The phrase "student profile vector graph" is best implemented as a hybrid representation rather than a single technology. Socra has three distinct needs: an interpretable per-topic state vector, a graph of educational relationships, and semantic embeddings for similarity/retrieval. They solve different problems.

Learner model: evidence first, derived state second



6.1 Per-topic state vector

For every student-topic pair, maintain a compact derived state. Example internal feature vector:

Feature	Example	Meaning
proficiency_estimate	0.64	Internal probabilistic/normalized estimate, not shown as "64% mastery"
evidence_confidence	0.81	How much/high-quality evidence exists
recent_trend	+0.12	Direction of recent performance
assistance_dependency	0.38	Degree to which success required deep help
recency_days	5	Time since strong evidence
first_attempt_rate	0.55	Unaided-ish first-response performance
retention_score	0.71	Performance on delayed/transfer items when available
misconception_distribution	{base_case: .42, trace: .10}	Probabilistic observation summary
evidence_count	14	Support for confidence calibration

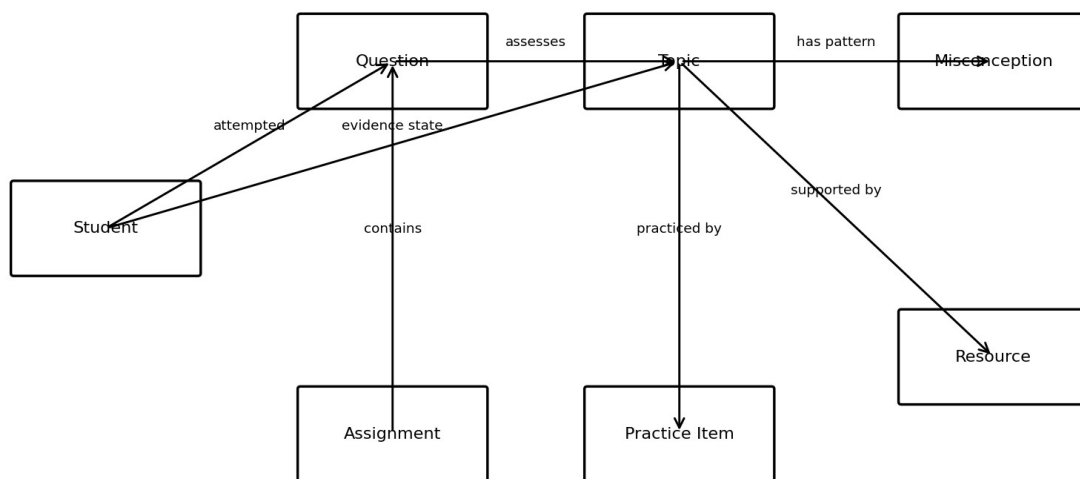
6.2 Recommended V1 estimator

For the pilot, start with an interpretable rule-based or logistic evidence model whose weights are explicit and validated. Bayesian Knowledge Tracing (BKT) and Logistic Knowledge Tracing (LKT) are natural later baselines because they model evolving student knowledge and can be interpretable. Deep Knowledge Tracing and transformer-based models can capture more complex sequences, but they require more data and make explanation/research attribution harder. Contemporary knowledge-tracing research continues to balance predictive accuracy with interpretability and use of process data.

Do not choose a deep learner model because the product uses AI elsewhere. The pilot may have too little data for a stable high-capacity student model. Start with transparent features; benchmark BKT/LKT; add neural models only when longitudinal sample size justifies them.

6.3 Learning graph

Logical learning graph - useful as a view, not necessarily a separate graph database in V1



The logical graph is valuable because the domain is relational: questions assess topics, topics have prerequisites, misconceptions attach to topics, resources teach topics, assignments contain questions, and students accumulate evidence on topics. In V1, this does not require Neo4j or another graph database. PostgreSQL tables for nodes/edges plus recursive queries or materialized views may be simpler and easier to operate. Introduce a graph database only if graph traversal becomes a real bottleneck or product requirement.

6.4 Semantic embeddings

Embeddings are appropriate for course-resource retrieval, similar-question search, clustering semantically similar student questions, and finding candidate examples. They are not a good sole representation of learner proficiency. An embedding is optimized for semantic geometry, not for calibrated educational state, evidence confidence, or causal interpretation.

- Embed course content chunks and question text for retrieval.
- Optionally embed de-identified Socra turns for offline clustering of common question themes.
- Do not store student names, university IDs, or unnecessary identifiers inside embedding text.
- Do not infer topic proficiency simply by cosine similarity between a student-chat embedding and a topic embedding.
- Version embedding model and regenerate indexes when model families change materially.

7. Data From Students, For Students

Student-facing data should answer "what should I work on next?" without becoming a surveillance dashboard or permanent ability label.

Student view	Source	Recommended presentation
Topic state	Learner state	Needs reinforcement / Developing / Consistently demonstrated
Trajectory	Recent evidence window	Improving / Stable / Needs attention
Why this recommendation	Evidence summary	"You needed several hints on base cases in two recent activities"
Practice suggestion	Topic + item selector	Targeted practice CTA
Recent history	Sessions/submissions	Resume/review
Post-assessment reflection	Closed assignment evidence	What changed and what to revisit

A student should be able to understand the basis of a recommendation at a human level. Avoid showing false precision such as 83.27% mastery unless the value has a defensible psychometric meaning and the UI clearly explains it.

8. Data From Students, For Faculty

Faculty analytics should aggregate learning evidence into instructional decisions. The default unit is the class, assignment, question, or topic, not the transcript of an individual student.

Metric	Computation	Instructional question
First-attempt correctness	correct first valid response / eligible first responses	Was the question initially understood?
Final correctness	correct final submissions / submissions	Did students eventually solve it?
Guided recovery	correct after Socra + revision / Socra users who revised	Did guidance coincide with recovery?
Mean intervention depth	mean max hint level per task	How much support did this concept require?
Misconception prevalence	unique students with canonical misconception evidence / eligible students	What are they misunderstanding?
Retry improvement	later score minus first score	Are students learning across attempts?
Delayed/transfer performance	performance on later linked items	Did understanding persist/generalize?
Unresolved concept	difficulty remains above configured threshold with adequate evidence	What needs reteaching?

8.1 Aggregation safeguards

- Define denominators in a data dictionary. "42% struggled" is meaningless unless "struggled" and the eligible population are explicit.
- Use minimum-n suppression or "insufficient data" for very small groups.
- Separate observed change from causal claims. "Correctness increased after guidance" is not the same as "Socra caused the increase."
- Do not use raw time-on-page as a direct measure of effort or learning.
- Provide confidence/coverage indicators when analytics depend heavily on model-inferred misconception labels.

9. Research Data Pipeline

Research datasets should be generated from production data through an explicit, versioned export pipeline. They should not be analysts running arbitrary queries against production tables.

- Freeze a dataset specification: cohort, date range, assignments, events, outcome variables, exclusions, and pseudonymization rules.
- Map internal user IDs to research participant IDs in a separately protected mapping table.
- Join event, submission, AI, and learner-evidence tables using stable versioned keys.
- Create immutable dataset snapshots with dataset_version and generation code commit/hash.
- Record model, prompt, policy, assignment, and question versions so AI-assisted conditions are reconstructable.
- Validate row counts and study-condition balance before release to analysis.
- Audit every research export.

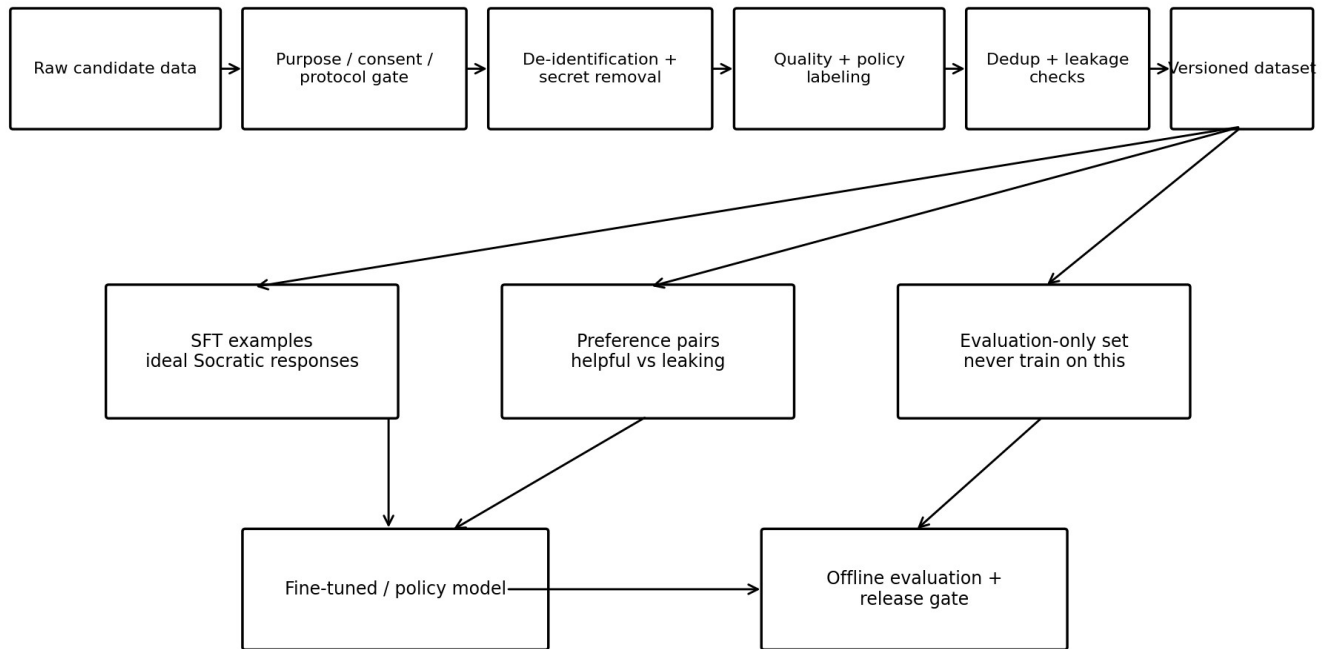
9.1 Train/test leakage for educational research

When evaluating predictive learner models, split data by student when the goal is generalization to unseen students. Randomly splitting individual interactions can put the same student in training and test sets, inflating performance through identity/history leakage. For temporal prediction, use time-aware splits that ensure future interactions do not leak into features computed for earlier predictions. For question generalization, consider item-level or course-cohort holdouts as a separate experiment.

10. Model Training and Fine-Tuning Data Pipeline

Production data and model-training data should be separate governance domains. OpenAI documents supervised fine-tuning as training on realistic examples in JSONL/chat format, but having the technical ability to fine-tune does not mean raw student interactions should be used automatically. For Socra, model improvement should begin with evaluation and prompt/policy engineering. Fine-tuning should happen only when there is a clear measurable failure mode that a curated dataset can address.

Model-improvement pipeline: curated, gated, reproducible



10.1 Candidate training datasets

Dataset	Example	Purpose	Pilot recommendation
Expert Socratic demonstrations	student prompt + ideal non-solution-bearing response	SFT for tutoring style/policy	Strongest first dataset
Leakage preference pairs	safe response vs answer-leaking response	Preference optimization / evaluator	Build early
Misconception labels	de-identified context -> canonical label	Classifier/fine-tuning	Useful after taxonomy stabilizes
Faculty authoring examples	brief -> reviewed scaffold/rubric	Authoring quality	Low privacy risk if no student data
Synthetic adversarial prompts	jailbreak/direct-answer attempts	Policy evaluation/training	Strong
Raw student chats	unaltered conversations	Potential future corpus	Do not train directly in V1

10.2 Curation gates

- Purpose gate: identify exactly which model behavior the example is intended to teach.
- Authority/protocol gate: confirm the project is permitted to use the data for model development, distinct from merely operating or studying the product.
- De-identification gate: remove direct identifiers and context that can reasonably re-identify a student where possible.
- Secret/IP gate: remove credentials, private repository material, hidden tests, instructor-only solutions, and licensed material not authorized for training.
- Quality gate: expert review of target response/label. A model-generated label is not automatically gold truth.
- Deduplication gate: near-duplicate conversations must not dominate the dataset.
- Split gate: assign examples to train/validation/test before iterative tuning; protect the evaluation set from training contamination.
- Version gate: dataset manifest includes source, transformations, reviewer, timestamps, code version, and exclusions.

10.3 What not to train on by default

- Names, email addresses, UB person numbers, or other direct identifiers.
- Raw grades tied to identity when unnecessary for the model objective.
- Private student disclosures unrelated to learning.
- Full raw transcripts merely because they are available.
- Hidden tests or instructor solution keys in a way that could cause future leakage.
- Data from participants whose approved research/consent conditions do not allow model-improvement use.
- The final held-out evaluation set.

11. Privacy, FERPA, and University Data Governance

UB's 2026 Access to Student Information policy describes educational records as records directly related to a student and maintained by the university or a party acting on its behalf, and explicitly includes student databases, papers, exams, correspondence, and similar records. UB also states that grades and courses are non-directory information. Socra data such as submissions, grades, learner profiles, and identifiable AI interactions therefore need to be treated as protected educational information within the university governance framework, subject to the actual pilot approvals and institutional guidance.

FERPA does not mean instructors may never see student information. The important product rule is legitimate educational purpose and least necessary access. Socra should provide faculty the information required to teach and intervene while not making raw private AI conversations the default analytics surface.

11.1 Privacy classes

Class	Examples	Controls
Identity	name, email, university identifier	Strict RBAC, separate from research IDs
Educational record	submission, grade, learner state	Course-scoped access, encryption, audit
Sensitive conversation	Socra messages, code snippets	Restricted access, retention policy, no faculty transcript dashboard
Aggregate	topic difficulty with sufficient n	Faculty access; suppression rules
Research pseudonymous	participant ID + events/outcomes	Research role, export audit
Model-training curated	de-identified approved examples	Separate storage/project and dataset manifest

11.2 Provider data boundary

OpenAI currently states that API inputs and outputs are not used to train OpenAI models by default unless the organization explicitly opts in. The platform also documents default abuse-monitoring retention that may include prompts/responses for up to 30 days, subject to endpoint and account controls. Socra should independently persist the minimum application state it needs, review the retention behavior of every API endpoint used, and not opt into provider data sharing for student data without explicit institutional authorization.

12. Recommended Physical Data Architecture for V1

Store	What lives there	Why
PostgreSQL primary	users, courses, assignments, versions, submissions, grades, evidence, learner states	Transactions, constraints, joins, auditability
Postgres outbox / queue	pending event messages/jobs	Reliable event handoff
Object storage	course files, large submission artifacts, research exports	Cheap durable blobs
Vector index	course chunks, questions, approved semantic artifacts	RAG/similarity
Analytics schema/materialized views	question/topic/course aggregates	Fast dashboards, stable metric definitions
Research snapshot area	immutable pseudonymous datasets	Reproducibility and access separation
Training dataset bucket/project	curated JSONL/preferences/evals only	Hard separation from raw production logs

A separate data warehouse is optional for the pilot. PostgreSQL can support the expected CSE 115/116 scale if schemas and indexes are designed properly. Introduce a warehouse when cross-course scale, heavy BI workloads, long retention, or institutional integrations justify the operational complexity.

13. Core Schema Sketch

learning_evidence

evidence_id, user_id, topic_id, question_id, type, numeric_value/json_value, confidence, weight, source_event_ids, detector_version, observed_at

learner_topic_state

user_id, topic_id, proficiency_estimate, confidence, trend, assistance_dependency, evidence_count, last_evidence_at, estimator_version

ai_request

request_id, user_id, mode, model, prompt_version, policy_version, input_tokens, cached_tokens, output_tokens, latency_ms, status, trace_id

misconception_observation

observation_id, user_id, topic_id, misconception_id, confidence, source_session_id, model_version, reviewed_status

question_aggregate_daily

course_id, question_id, date, eligible_n, first_attempt_correct_n, final_correct_n, socra_user_n, recovered_n, avg_intervention_depth

research_event

participant_id, event_name, event_time, condition, course_id, assignment_id, question_id, payload_version, approved_payload

dataset_manifest

dataset_id, purpose, query/code_version, source_window, inclusion_rules, exclusion_rules, deidentification_version, generated_at, checksum

14. Data Quality and Lineage

- Use schema validation at ingestion. An event with missing assignment_version should fail or be quarantined rather than silently accepted.
- Use referential constraints for authoritative IDs.
- Monitor event lag, duplicate rates, missing-event rates, and aggregate reconciliation against transactional counts.
- Create lineage links from dashboard metric -> aggregate row -> evidence records -> source events/transactions.
- Backfill jobs must be versioned and idempotent.
- When a bug changes a metric definition, create a new metric version rather than silently rewriting published research outputs.

14.1 Suggested data quality SLIs

SLI	Target / alert idea
Submission/event reconciliation	>=99.9% submitted snapshots have corresponding submission event
AI request accounting	>=99.9% completed requests have usage/status row
Event freshness	p95 evidence processing lag < 5 min for dashboards
Aggregate freshness	course dashboards updated within 15 min during pilot
Orphan evidence	0 evidence rows with invalid topic/question FK
Duplicate event processing	0 net double-counted aggregates after replay
Research export checksum	manifest checksum + row count verified before analysis

15. Misconception Pipeline

Misconceptions are especially valuable and especially easy to overclaim. Use a canonical taxonomy plus probabilistic observations.

- Faculty/research team defines or approves canonical misconception IDs for initial topics.
- Rules or an LLM proposes misconception observations from code, answer, and conversation context.
- Each observation stores confidence and detector version.
- Low-confidence novel patterns enter a clustering/review queue rather than immediately appearing as a named faculty metric.
- Faculty dashboard aggregates unique students/evidence using a documented threshold.
- Human review can promote a recurrent cluster into a canonical misconception for future detection.

16. Feature Pipelines and Online vs Offline Consistency

If Socra later uses predictive models in real time, feature definitions must be consistent between training and inference. A classic failure is training on a feature computed with future data and then trying to serve it online.

Feature	Online computation	Offline training snapshot
recent_correct_rate	last N eligible attempts before now	same event cutoff before prediction timestamp
days_since_success	current time - last success	prediction_time - prior last success
hint_depth_mean	eligible prior sessions only	same historical window
topic_prerequisite_state	latest states available before request	as-of join, never future state
question_difficulty	frozen/versioned estimate	same version used for training

Every training row needs an "as-of" timestamp. Features must be computed only from information available before that timestamp. This is how Socra avoids temporal leakage.

17. Retention, Deletion, and Recomputability

Retention must be set with the university/research team before launch. Different data classes should have different schedules. A single "keep everything forever" policy is not appropriate.

Data class	Retention design question
Raw Socra messages	How long are transcripts needed for student history, research, debugging, or disputes?
AI request logs	Can content be removed while retaining usage/latency metadata?
Submissions/grades	What is the university/course records policy?
Learning evidence	How long is it needed for profile/research; can it be de-identified?
Learner state	Derived and recomputable; deletion should cascade/recompute
Research snapshots	Per IRB/study data-management plan
Training datasets	Separate lifecycle; remove source examples if policy requires and technically feasible

Deletion should not merely remove a dashboard row while leaving embeddings, cached features, and derived profiles intact. Maintain a data lineage map so deletion/de-identification jobs can locate downstream representations. For trained model weights, deletion is more complex, which is another reason to gate training datasets carefully before training rather than promising perfect post-hoc removal from weights.

18. Education Data Standards and Interoperability

1EdTech Caliper Analytics is directly relevant to Socra because it defines a common vocabulary and event-oriented framework for learning activity, including assessment, grading, feedback, session, and tool-use profiles. Socra does not

need to make every internal event a Caliper event in V1, but designing an internal event vocabulary that can map to Caliper later will make future LMS/institution integrations easier. 1EdTech also emphasizes stewardship, security/privacy, ownership, and transparency in learning analytics.

- Use internal event names optimized for product development, but document a future Caliper mapping.
- Keep course/topic identifiers stable enough to map to external standards or competency frameworks later.
- For future LMS integrations, consider 1EdTech LTI/OneRoster/QTI/CASE alongside Caliper rather than inventing proprietary interfaces for every institutional system.

19. Security Controls for the Data Pipeline

- Encrypt data in transit and at rest using the hosting platform's managed controls.
- Use least-privilege service identities. The analytics worker should not need administrative roster permissions.
- Separate production, staging, research-export, and training-dataset credentials/projects.
- Audit privileged reads of sensitive conversations and every research/training export.
- Do not put hidden tests, provider API keys, or authentication tokens into analytics events.
- Redact secrets from code/run logs before long-term storage.
- Use row/course authorization at the API layer; never rely only on client-side filtering.
- Back up authoritative data and test restores.

20. Build Plan for the Data Platform

Phase	Build now	Defer
P0: pilot foundation	Postgres schemas, outbox, event taxonomy, evidence ledger, aggregates, pseudonymous research IDs, AI usage logging, audit logs	Kafka, graph DB, feature platform
P1: learning intelligence	topic state estimator, misconception taxonomy, practice selector, faculty question/topic aggregates	deep knowledge tracing
P2: research maturity	immutable dataset manifests, as-of feature snapshots, evaluation datasets, reproducible notebooks/pipelines	automated institution-wide warehouse
P3: model improvement	expert demos, preference labels, de-identification pipeline, fine-tuning/eval manifests	training directly on raw chats
P4: scale	warehouse/lakehouse, graph service if needed, online feature store, cross-course learner models	premature complexity

21. Concrete Architecture Decisions Recommended for Socra

- Use PostgreSQL as the system of record and initial analytics/evidence store.
- Use a transactional outbox rather than introducing Kafka for the first university pilot.
- Create an immutable learning_evidence table and a separately recomputable learner_topic_state table.
- Represent the curriculum as explicit topics/prerequisites and evidence relations; implement the "graph" as relational edges/views in V1.
- Use vectors for semantic retrieval, not as the sole learner model.
- Start learner-state estimation with transparent weighted/logistic evidence; benchmark BKT/LKT later.
- Compute faculty statistics deterministically; use LLMs only to summarize computed data.
- Pseudonymize research datasets and version every export.
- Do not use raw pilot conversations for fine-tuning by default. First create expert-reviewed and synthetic Socratic datasets.
- Separate train, validation, and evaluation datasets by student/source and protect the evaluation set from contamination.
- Keep raw conversations out of ordinary faculty analytics.

- Version model, prompt, policy, assignment, question, taxonomy, estimator, metric, and dataset semantics.

22. End-to-End Example: One Recursion Question

1. **Student opens Question 4:** assignment_opened/question_viewed events emitted.
2. **Student runs solve(5):** code_run record stores snapshot hash, runtime status, public output. Event links run to question version.
3. **Student asks "why does this never stop?":** ai_session + ai_request store model/prompt/policy versions; current code is sent to protected Socra.
4. **Socra asks the student to trace values:** response stores intervention_level=2. Raw transcript remains in conversation domain.
5. **Student traces 5,3,1,-1 and revises:** events record answer/code change and another run.
6. **Submission becomes correct:** immutable submission snapshot and grading result created.
7. **Evidence normalizer runs:** creates correctness, recovery_after_guidance, hint_depth, recursion/base-case evidence.
8. **Misconception detector runs:** creates probabilistic observation: assumes decreasing sequence reaches zero, confidence=.88.
9. **Learner state updates:** recursion/base-case state changes using current estimator_version. Old evidence remains.
10. **Faculty aggregate updates:** question first-attempt %, guided-recovery %, misconception prevalence, average intervention depth.
11. **Research snapshot later includes it:** participant ID, condition, versions, outcomes, approved event features. Direct identity omitted.
12. **Training pipeline does nothing automatically:** the transcript is not a fine-tuning example unless a later governed curation process selects/de-identifies/reviews it.

23. Research Basis and References

The architecture above combines standard data-engineering practice with education-specific learning analytics and knowledge-tracing research. Key sources checked for this paper:

- [University at Buffalo - Access to Student Information / FERPA policy \(updated 2026\)](#)
- [University at Buffalo - Standards for Protecting University Data](#)
- [U.S. Department of Education - Personally Identifiable Information for Education Records](#)
- [1EdTech - Caliper Analytics](#)
- [1EdTech - Learning Data & Analytics Key Principles](#)
- [OpenAI - Data controls in the API platform](#)
- [OpenAI - Supervised fine-tuning guide](#)
- [NIST - AI Risk Management Framework](#)
- [Abdelrahman, Wang, Nunes - Knowledge Tracing: A Survey, ACM Computing Surveys](#)
- [Piech et al. - Deep Knowledge Tracing, NeurIPS 2015](#)
- [Lu, Tong, Cheng - Advanced Knowledge Tracing with Process Data, JEDM 2024](#)
- [Pavlik et al. - Logistic Knowledge Tracing Tutorial, EDM 2024](#)
- [Wu et al. - Graph-based Student Knowledge Profile, SIAM SDM 2024](#)

Important: this paper is an engineering/product architecture recommendation, not legal advice or a substitute for UB Information Security, Registrar, IRB, or institutional research-data review. The research team should reconcile the final retention, access, consent, and model-improvement rules with the approved study protocol and current university policy before launch.

Appendix A. Suggested Socra Event Dictionary

Event	Minimal payload
assignment.opened	assignment_id, version, course_id
question.viewed	question_id, version
draft.saved	draft_version, content_hash, byte_count
code.run_completed	run_id, language, status, duration_ms, public_test_summary
socra.prompt_sent	session_id, mode, message_id, question_id
socra.response_completed	response_id, model, prompt_version, policy_version, intervention_level, latency_ms, token_usage
socra.resource_retrieved	resource_id, chunk_id, rank, retrieval_model_version
submission.completed	submission_id, attempt_no, snapshot_hash
grade.finalized	submission_id, score, rubric_version, grader_type
misconception.observed	misconception_id, confidence, detector_version
learning_evidence.recorded	evidence_id, topic_id, type, value, source IDs
learner_state.changed	topic_id, previous_state, new_state, estimator_version
practice.item_answered	item_id, correctness, attempts, assistance
faculty.analytics_viewed	view_type, course/assignment/question scope
research.export_generated	dataset_id, dataset_version, row_count, checksum

Appendix B. Metric Definition Rules

- Every percentage has an explicit numerator and denominator.
- Every metric declares whether attempts are per-student, per-question, or per-submission.
- Every aggregate declares its date/time boundary and timezone.
- Every metric based on model labels declares the confidence threshold and detector version.
- Every learner-state snapshot declares estimator version.
- Every research table declares whether rows are interaction-level, student-level, question-level, or student-question-level.
- Every causal-looking phrase in the UI is reviewed to ensure the underlying design supports causal inference.

Appendix C. Training / Research Dataset Manifest Template

dataset_id
name
purpose
owner
source systems
source date range
eligible cohort
IRB / policy basis
fields included
fields excluded
de-identification transformations
labeling method
quality-review procedure
deduplication method
train/validation/test split rule
known limitations
code/query version
model/prompt versions represented
record count
checksum
created_at
retention / deletion rule